

FISPACT-II Advanced (API) exercises

UKAEA

June 12, 2025

These advance exercises include methods using both the executable and the FISPACT-API. Both do not need to be completed, you may follow your preferred approach, but it may be useful to complete both so the merits of each approach can be compared. The text and resources assume that the Python3 API is being used, but the exercises may be completed in any of the other supported languages (Fortran, C, C++).

1 Exercise multiflux

This exercise examines the ability of FISPACT-II to perform sequential irradiations with differing flux spectra, using a series of chained simulations. Thus, the output from one simulation will be used as the input for the next simulation. For the sake of simplicity, we will only use two different energy spectra, but in reality this principle can be applied to any number of iterations, whereby the spectra is changing.

In the working directories for this example, '*advanced/multiflux_standard*' and '*advanced/multiflux_API*', two fluxes files can be found, *fluxes.1* and *fluxes.2*, representing the spectra at two different times during an irradiation. The spectra are based on a deuterium-tritium (D-T) reaction inside a tokamak - notice the 14 MeV peak. We first plot the two spectra for comparison, as shown in figure 1. Are you able to reproduce this plot?

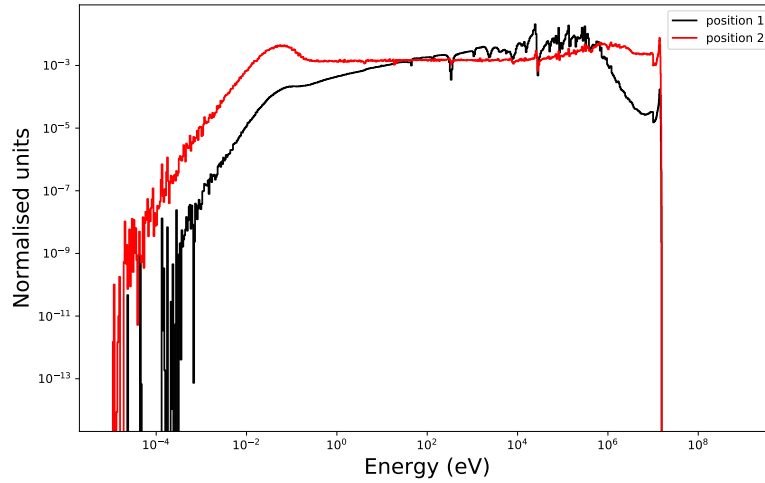


Figure 1: The incident particle spectra as a function of energy in 709 group, for D-T reaction inside a tokamak at time 1 (black) and time 2 (red). Spectra are normalised to unity.

We will irradiate 1 gram of Tungsten following an irradiation of $1.1 \times 10^{14} \text{ n cm}^{-2} \text{ s}^{-1}$ for 5 minutes (time 1) with the first spectrum, followed by an irradiation of $1.3 \times 10^{14} \text{ n cm}^{-2} \text{ s}^{-1}$ for 10 minutes (time 2), followed by a period of cooling for 10 years in 1 year increments. In order to do this we need to run two separate simulations, the first input file is given in the exercise directory, aptly named *first.i*. The second input file does not yet exist as

```

TIME 0.000ps      INTERVAL 1 INTERVAL TIME      0.000E+00 SECS
W 180      741800  3.93086E+21  1.17458E+00
W 182      741820  8.68065E+23  2.62270E+02
W 183      741830  4.68755E+23  1.42406E+02
W 184      741840  1.00368E+24  3.06582E+02
W 186      741860  9.31287E+23  2.87567E+02
TIME 5.000 m      INTERVAL 2 INTERVAL TIME      5.000E+00 MINS
H 1         10010  2.61857E+11  4.38226E-13
H 2         10020  8.28090E+09  2.76954E-14
H 3         10030  4.67742E+08  2.34258E-15
He 3        20030  1.24984E+02  6.25951E-22
He 4        20040  1.30286E+11  8.65941E-13
Hf176       721760 1.64155E+07  4.79591E-15
....

```

Figure 2: The expected output TAB1 file produced after running the first example.

this depends on the output of the first run. There are two approaches to doing this, by hand, or automate it using the API.

1.1 Standard approach

We will leverage the *TAB1* keyword to enable an additional output file to be written in tabular format. This keyword should be placed in the initialisation phase of the input file to enable all outputs to be written to the file. The first input file *first.i* already has this keyword enabled. After running this simulation a *first.tab1* file should be produced, which should look like figure 2. We should now be able to construct a second input file, *second.i*, which uses the nuclides in the final inventory from the output of the first, but now using the second spectra as our fluxes source. Are you able to do this?

Hint: The third column in the atoms inventory, which can be directly plugged into the input file via use of the *FUEL* keyword. Note that we must remove the whitespace in nuclide names in order for FISPACT-II to correctly process them.

If you did manage to do this by hand, did you find this an easy process? We will try the automated approach now and compare the results.

1.2 Using the FISPACT-API

To perform sequential irradiations with different spectra and/or particles the resulting nuclides from one irradiation need to be used as input for the next. When using the FISPACT-API this involves extracting the inventory from the OutputData object and using it in an InputData object for a new irradiation. Each nuclide and its number of atoms should be extracted from the last timestep of the previous irradiation. These can be used as input for *setfuel* in subsequent calculations. The script *multiflux_API/multiflux.py* currently only performs an irradiation with *fluxes.1*. Can you complete the script to take the output of the first irradiation as the input of the second with *fluxes.2*?

The following extract from the API manual may be useful:

In the examples a function is shown which will find the activity for a given nuclide at a given inventory index. These functions are generic and could be simply modified to output any quantity in the output nuclide data struct.

```

...
# extract activity - generic routine
def extract_activity_from_nuc(o, inv_index, nuclide_zai):
    activity = -1.0
    # check if nuclide present at a given step
    if (o.findinventoryexists(inv_index, nuclide_zai)):

```

```

# get index of nuclide in inventory nuclide list
nuclide_index = o.findinventoryindex(inv_index, nuclide_zai)
# get nuclide list at given step
nuclide_list = o.getinventorynuclides(inv_index)
# get heating value for that nuclide from inventory
activity = nuclide_list[nuclide_index].activity
return activity
...

```

2 Exercise Enrichment

This exercise will demonstrate how to use FISPACT-II to irradiate samples on non-natural abundance. The natural abundances used by FISPACT-II is hard-coded and taken from NIST, this can be output with PRINTLIB.

```

...
PRINTLIB 9
...

```

2.1 Standard approach

The MASS keyword sets a material by weight percent (assuming natural abundance) and the FUEL my number of atoms. So to change the ratio of isotopes of a given element the FUEL keyword must be used, however the number of atoms for a given mass of material must be known. In this exercise enriched and unenriched 1kg Li samples will be irradiated for 1 hour, using a typical Fusion breeder blanket neutron spectrum, and the amount of tritium produced compared.

The input (*.i*) file for the unenriched case is can be found at '*advanced/enrichment_standard*'. This should be executed first. How much tritium is produced in this irradiation?

Now construct a new input file and perform the same irradiation but using a 60% enriched ^6Li sample. The unenriched input file can be used as template and the same *files* file and *fluxes* file can be used. How much tritium is produced in this case?

Hint: The unenriched case can give the total Li atoms in a in kg and the ^6Li and ^7Li atoms in a kg. What is the minium FISPACT-II input file required to be able to extract this information? Do you need irradiation steps?

2.2 API methods

The FISPACT-API exposes the natural abundance data via the *elementaldata* methods. With these it is possible to extract and change any piece of hardcoded elemental data used by FISPACT-II. With these methods the natural abundance can be altered *on the fly*. A Python script called *enrichment.py* can be found in '*advanced/enrichment_API*'. This currently performs the unenriched irradiation, can you expand the script to included the unenriched case using the ?

Hint: As the *elementaldata* methods change the abundance data used by FISPACT-II, do you need to use the *setfuel* methods?

3 Further API exercises

If additional exercises are desired you can attempt to replicate the *extended* exercisers with the API. We recommend starting with *groupconvert* and *READGG*. Note that self shielding and pathways are currently not supported in the API approach.